AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_HWSW_task_3 Version : 1	

HW/SW IP KOMPONENT-
PROTOTYP AV AVSTÅNDSMÄTARE MED EN ULTRALJUDSENSOR

Darwin Rengifo

2019-02-24

Sammanfattning:
Denna rapport beskriver en förstudie och planering för avståndsmätare projektet med ultraljudsensoren "HC-SR05" med användningen av IP komponenter och Qsys-vektyget för att integrerar hårdvaran och mjukvaran interface genom läsningen och skrivningen på Avalon bus. Sensorn skickar ultraljud signaler för att upptäcka objekt som ligger i framfört sensorn på en intervall av 2 till 450 cm när dessa ultraljud signaler upptäcka objektet kommer de tillbaka till sensorn för att beräkna avståndet. Det visar mätningen resultaten på sjusegmentsdisplayerna som finns på utvecklingskortet och VGA-skärmen som visar färger röd, grön, blå och vit för olika avstånd. Denna rapport innehåller aktivitets- och tidplan, projektkostnad, kravspecifikations, projektarkitektur och beskrivning av utvecklade komponenter. För att slutsats sammanfattningen detta projekt visar användningen av ultraljudsensoren för att göra mätningar och validerar resultaten igenom gränssnitt av sjusegmentsdisplayerna och VGA-skärmen.

Innehåll

1. INLEDNING 4
2. KRAVSPECIFIKATION FRÅN KUND 5
3. PROJEKPLAN 8
4. HÅRDVARAN ARKITEKTUR FÖR INGÅENDE KOMPONENTER. 9
4.1 Komponenter 9
4.1.1 DE10_Lite_Sensor 9
4.1.2 Sensor IP 10
4.1.2.1 Trigger_cont 10
4.1.2.2 Echo_cont 10
4.1.2.3 Bcd_converter 10
4.1.2.4 Sju_seg_displayer 10
4.1.3 Vga_IP 10
4.1.3.1 VGA_controller 10
4.1.3.2 Vga_ram 10
4.1.3.3 Clk_25mhz 10
4.1.4 Timer_hw_IP 10
4.1.4.1 Timer_top 11
4.1.5 PLL 11
4.1.6 System ID 11
4.1.7 On-Chip RAM 11
4.1.8 JTAG UART 11
4.1.9 SDRAM Controller 11
5. KONSTRUKTIONS BESKRIVNING 12
5.1 Systemarkitektur 12
5.2 Delsystem 14
5.2.1 Toppnivån 14
5.2.1.1 Systemarkitektur 14
5.2.2 Sensor IP – komponent 16
5.2.3 Trigger - komponent 17
5.2.3.1 Systemarkitektur 17

5.2.4 Echo - komponent ... 18
5.2.4.1 Systemarkitektur ... 18
5.2.5 Bcd konverter - komponent ... 19
5.2.5.1 Systemarkitektur ... 19
5.2.6 Sju_seg_displayer - komponent ... 19
5.2.6.1 Systemarkitektur ... 19
5.2.7 Vga IP – komponent ... 20
5.2.7.1 Systemarkitektur ... 20
5.2.8 Vga kontroller - komponent ... 21
5.2.8.1 Systemarkitektur ... 22
5.2.9 PLL – komponent ... 23
5.2.10 Ram kontroller - komponent ... 23
5.2.10.1 Systemarkitektur ... 23
5.3 Ultraljudsensor hc-sr05 ... 24
5.4 I/O pinnar ... 25
6. ANALYS ... 26
6.1 Frekvens och storlek före optimering ... 26
6.1.1 Max frekvens för hela konstruktionen ... 26
6.1.2 Storlek för hela konstruktionen och delsystem ... 26
6.2 Effektförbrukning uppskattning ... 27
6.3 Tisdvalidering ... 28
7. VERIFIERING AV TESTPROTOKOLL ... 29
8. VALIDERING KONSTRUKTIONEN ... 30
8.1 Testprotokoll för validering ... 30
8.2 Validering film ... 30
9. MJUKVARAN ARKITEKTUR ... 31
9.1 Systemarkitektur ... 31
9.2 Delsystem ... 32
9.2.1 Minneskarta ... 33
9.3 Drivrutiner ... 33
9.3.1 VGA färger ... 33
9.3.2 VGA-drivrutiner ... 33
9.3.3 Timmer-drivrutiner ... 35

9.3.4 Sensor-drivrutiner 35
10. OPTIMERING 36
11. REFERENSER 36

1. INLEDNING

Denna rapport beskriver som bygger en avståndsmätare med ultraljudsensor och alla IP komponenterna som kommer att bygga för att se resultat av visualisering på sjusegmentsdisplayerna som finns på utvecklingskortet och VGA-skärmen som ska det visar färger röd, grön, blå och vit för olika avstånd. Det kommer att validerats på ett lämpligt FPGA-kort och VGA-skärmen.

2. KRAVSPECIFIKATION FRÅN KUND

Tabell 1. Kravspecifikation från kund

Krav	Beskrivning	Utfört
		Ja/nej
001	Kundprojektets arbetsnamn är VGA_IP	
002	Skapa en IP komponent med VGA-komponenten och återanvänd tidigare VHDL- och C-projekt.	Ja
003	Krav på konstruktionskoden: 1)"filhuvud" i C-filen, 2) Namn på variabler m.m. ska vara tydliga, 3) Rikligt med kommentarer i koden, 4) Frivilligt: Kontroll av gränsvärden ska göras, t.ex. att värdet inte ligger utanför skärmen. Om detta inträffar ska ett felmeddelande skrivas ut till systemterminalen (Nios II "console").	Ja
Funktionskrav		
004	Följande drivrutiner sparas under HAL och ska vara uppdaterade i VGA_IP_SW_sw.tcl filen). Drivrutinerna kan kopieras från tidigare utfört projekt i C-kursen eller från ett äldre projekt. Funktion: print_pix(unsigned int x,unsigned int y,unsigned int rgb); Funktionsbeskrivning: Skriver en pixel med färgen rgb på koordinaten (x, y). Alternativ lösning för print_pix; om funktionen behöver snabbas upp, kan den istället göras om till ett macro.	Ja

	Funktion: print_hline(unsigned int x_start, unsigned int y_start, unsigned int len, unsigned int RGB); Funktionsbeskrivning: Skriver en horisontell linje med färgen rgb och med längden len vilken startar på koordinaten (x_start, y_start). Funktion: print_vline(unsigned int x_start, unsigned int y_start, unsigned int len, unsigned int RGB); Funktionsbeskrivning: Skriver en vertikal linje med färgen rgb och med längden len vilken startar på koordinaten (x_start, y_start). Funktion: print_char(unsigned int x,unsigned int y,unsigned int rgb,unsigned int BG_RGB,char Character); Funktionsbeskrivning: Skriver tecknet "Character" med färgen rgb och med bakgrundsfärgen "BG_RGB" på koordinaten (x, y). Lägg till de ASCI som behövs för att skriva det som krävs. Det går att bestämma hur tecknen ska visas på skärmen, se bilaga, äldre projekt och på webben. Funktion: read_pixel_ram_int(unsigned int x_start, unsigned int y_start); return: unsigned pixel_data (obs! enbart de sista tre bitarna är intressanta). Funktionsbeskrivning: Läser pixel_data från pixel RAM (3 bitar, RGB) från adress x och y (beräknat från x och y). Funktion: clear_screen (int rgb); Funktionsbeskrivning: Denna funktion rensar skärmen genom att RGB-värdet "000"(svart) skrivs till alla pixlar på skärmen. Funktion: print_circle(unsigned int radie, unsigned int x_centrum, unsigned int y_centrum, unsigned int rgb); Funktionsbeskrivning: Skriver en cirkel med radien "radie" och färgen" rgb" på mittkoordinaten (x, y). Cirkeln ska fyllas med samma färg.	Ja
Valideringskrav		
005	Testprotokollet kan vara ofullständigt, men ska innehålla minst två testfall (HW/SW design report).	Ja
006	Skriv ditt namn på skärmen, fotografera skärmen och lägg in fotot som figur i rapporten (HW/SW design report).	Ja
007	Rapporten ska också innehålla kostnaden för konsultuppdraget. Företaget konstukören är anställd på fakturerar 600 SEK/timme för projektet. Totalt antal timmar * 600 SEK utgör den totala kostnaden.	Ja
Struktur på leveransen och dokumentationskrav		

008	Leverera följande struktur med zippade filer. VGA_IP (top library and the sub library) • VGA_IP_HW_hw.tcl • VGA_IP_SW_sw.tcl • hdl (HW), - ….vhd (VHDL code)	Ja
	• HAL (SW), - Inc (library for h files) - src (library for c files) • doc (documentations), - VGA_datasheet - Software manual for the programmers - SW/HW Design Report (standard report) • Examples - Ett exempel system som skriver ut ditt namn på skärmen - SOF och sopcinfo filen - Arkiverade projektet - C filen (main) • Projects (HW) - VGA_IP (Quartus) - VGA_IP_HW (Quartus)	Ja
Leveranskrav		
009	Leveransen ska ske till plattformen Itslearning. Leveransen ska vara en IP-komponent enligt strukturen från krav 004. Namnet på filen ska vara ”förnamn_efternamn_HWSW_task_3.zip”. Sista leveransdag se kursschema (för VG).	Ja

3. PROJEKPLAN

Aktivitets - och tidplan för ingenjörsjobbet kan ses i Tabell 2. Totalt uppskattas ingenjörsjobbet ta 80 timmar/600kr(timme), vilket ger en lönekostnad på 40 000 SEK.

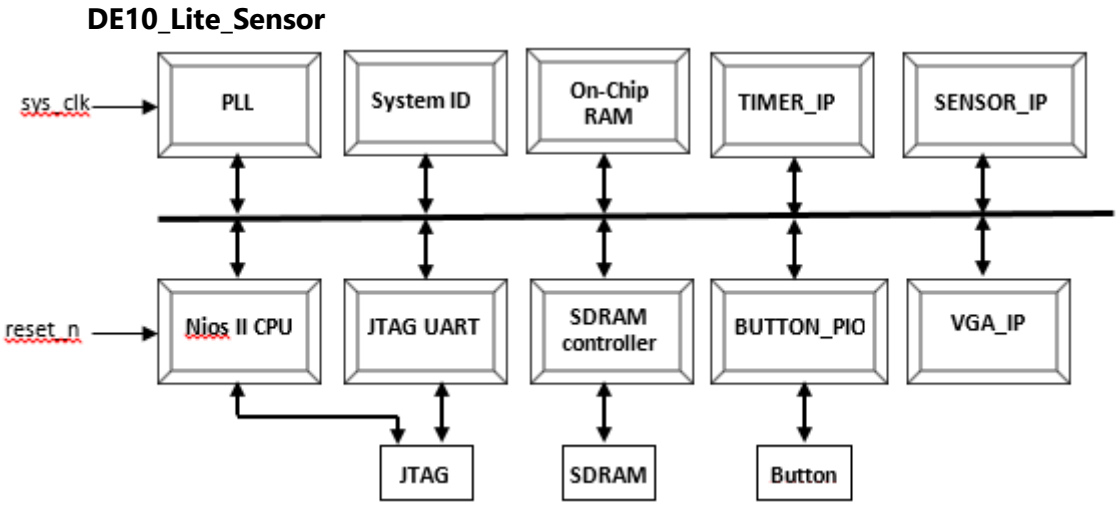
Tabell 2. Aktivitets- och tidsplan för ingenjörsjobbet

Aktivitet	Veckor				
	46	47	48	Timmar(h)	Kostnad(kr)
Kravspec./förstudie	13			13	7 800
Testprotokoll/ Drivrutiner	4	4	1	9	5 400
Arkitekturbeskrivning	5	3		8	4 800
Programmering av visuellt gränssnitt	8	12	8	28	16 800
Verifiering		2	2	4	2 400
Validering			2	2	1 200
Slutrapport		4	3	7	4 200
Summa tid	30	31	19	80	42 600

4. HÅRDVARAN ARKITEKTUR FÖR INGÅENDE KOMPONENTER.

HC-SR05-sensorn är en modul som innehåller ett par transducer som används tillsammans för att bestämma avståndet mellan sensorn och ett objekt som placerats framför den. En transducer avger ultraljud signaler och den andra tar upp studs för den vågen. Den tid som det tar ljudvågen att gå och återgå till ett objekt kan användas för att beräkna avståndet mellan ljudets ursprung och objektet.

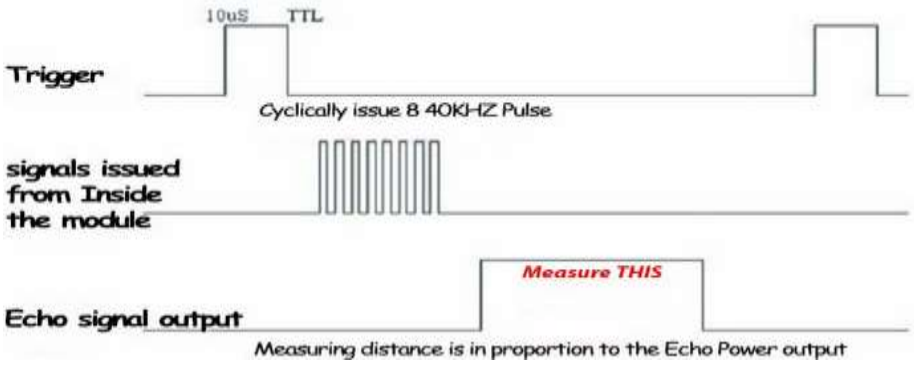
I figur 1 visas den arkitektur som används i projektet. De komponenter som ingår finns med i figuren.



Figur 1. Systemarkitektur, blockschema FPGA

4.1 Komponenter

I projektet ska flera komponenter utvecklas. Det behövs komponenter för att läsa information från sensor HC-SR05, räkna ut avstånd och visa detta på sjusegmentsdisplayer på utvecklingskortet samt visa färger på skärmen för att validera. I Figur 2 visas den diagrammet för tidläsning av ultraljudsensor.



Figur 2. Ultaljudsensor timing diagram

4.1.1 DE10_Lite_Sensor

Toppfilen som kommer att instansierar QSYS-interface.

4.1.2 Sensor IP

Filen som skapar Avalon bus interface av läsning/skrivning även instansierar alla komponenter som använder för att mäta avstånd med ultaljudsensor. Skapa rutin för att beräkning avstånd i centimeter.

4.1.2.1 Trigger_cont

Den ”trigger” pin får en aktivering puls av 10us från mikrokontroll genom denna puls indikerar till sensorn att måste börjar med avståndsmätaren.

4.1.2.2 Echo_cont

Denna komponent kontrollerar ”echo” räknaren. Räknar hur många klockpulser det finns på en ”echo” period. Igenom ”echo” pin, detta sensorn returnerar till mikrokontroll en puls vars bredd är proportionell till tiden som tar ljudet att resa från transducer till objekt och sen återgå till sensorn

4.1.2.3 Bcd_converter

Denna komponent omvandlar avstånd binär värde till bcd värde för att visa efter det på 7-segment display. Metoden shift-and-add-3 kommer användas.

4.1.2.4 Sju_seg_displayer

Styrprogram för 7-segment display på FPGA kort för att visar avståndet i centimeter.

4.1.3 Vga_IP

Filen som skapar Avalon bus interface av läsning/skrivning även instansierar alla komponenter som använder för att rita på skärmen.

4.1.3.1 VGA_controller

Denna VGA drivrutin skapar bilden av 640x480, men använder ett RAM-minne av 320x24.

4.1.3.2 Vga_ram

Det är en "Dual Port Ram" med dubbel klock för att kan läsa och skriver information på minnet. Skrivning process på RAM minne är på 50mhz och läsning process från RAM minne är på 25mhz.

4.1.3.3 Clk_25mhz

Det är en PLL för att delar frekvensen de 50mhz till 25mhz för att kontrollerar frekvensen i vga_controller och vga_ram.

4.1.4 Timer_hw_IP

Filen som skapar Avalon bus interface av läsning/skrivning även instansierar komponent som använder för att ändrar hastighet frekvensen för att visualisering av data på skärmen.

4.1.4.1 Timer_top

Kontrollerar sekv ensen av reset-stop-start-read på hårdvaran.

4.1.5 PLL

En PLL finns tillgänglig i systemet för att distribuera varianter av systemklockan på 50 MHz till delar av systemet.

4.1.6 System ID

En komponent som innehåller information om hårdvaruversion.

4.1.7 On-Chip RAM

Intern styrenhet för SDRAM-minnet.

4.1.8 JTAG UART

Kommunikationsgränssnitt mellan USB Blaster och data/instruktionsbus. Används för kommunikation mellan PC och CPU för programmering och debuggning.

4.1.9 SDRAM Controller

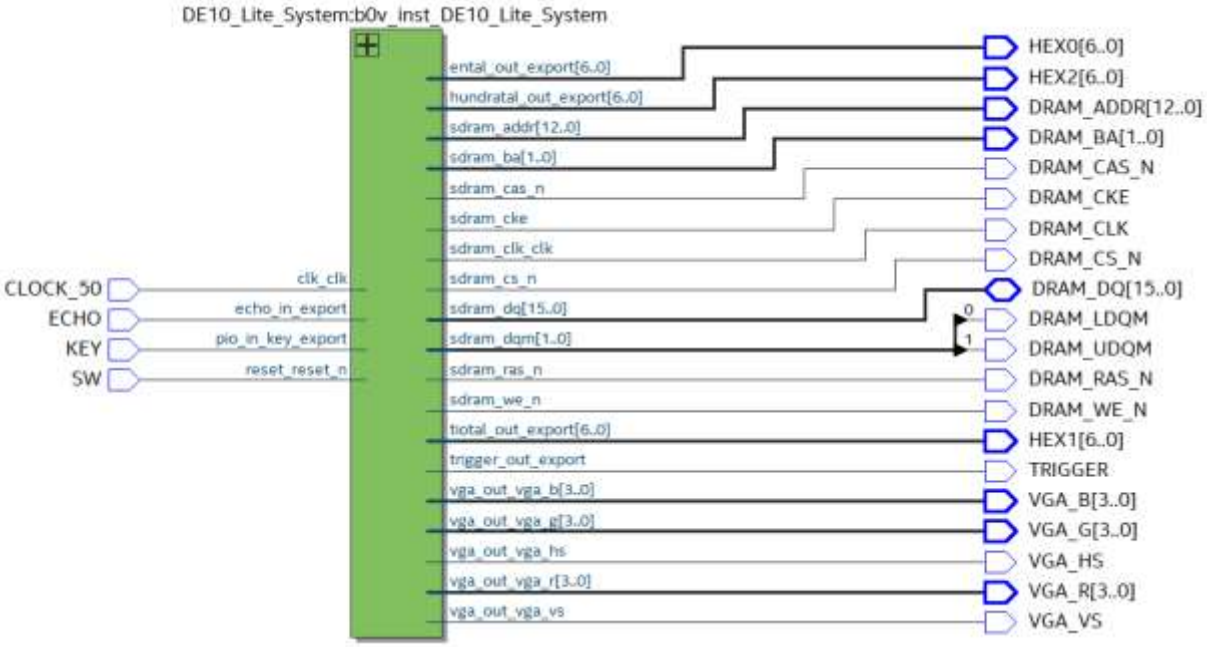
Extern styrenhet för SDRAM-minnet.

5. KONSTRUKTIONS BESKRIVNING

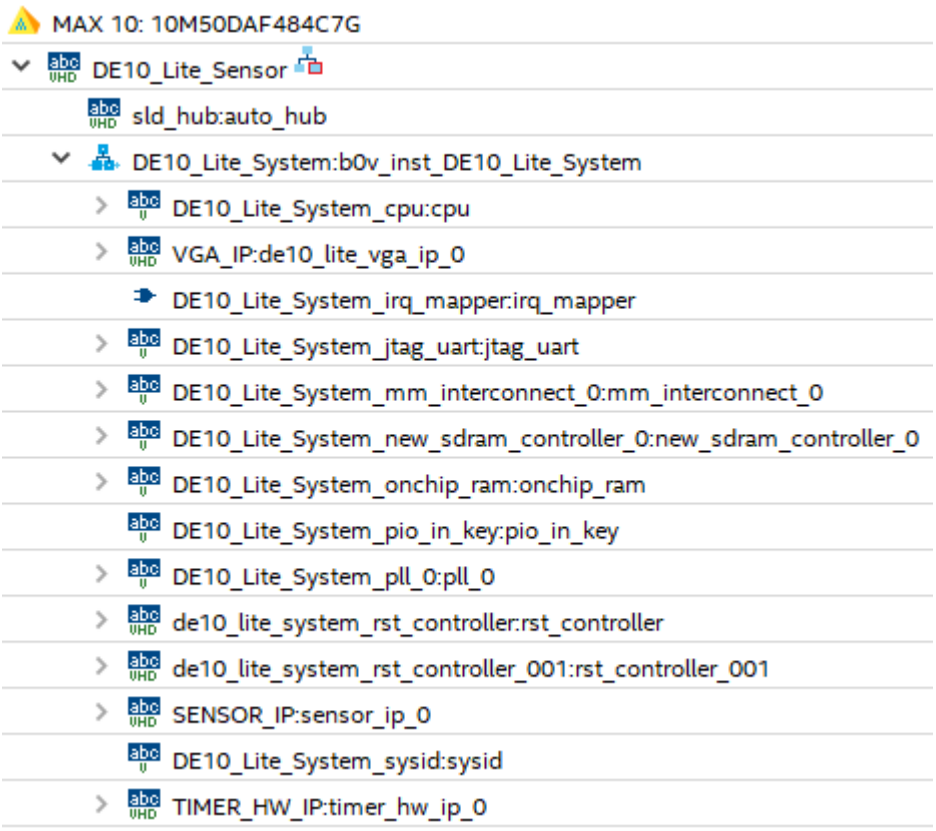
Programmet körs på kortet DE10-Lite är en robust hårdvara design plattform som byggde kring Altera MAX 10 FPGA med en Nios II-CPU som är anpassad för ingenjörsjobbet. DE10-Lite-kortet innehåller en 15-pin D-SUB-kontakt för VGA-utgång, VGA skärm för att visualisering resultaten, VGA-kabel för att skickar signalerna från DE10-Lite kortet till VGA skärm, en ultraljudsensor HC-SR05 för att göra avståndsmätare testen och en protoboard för att koppla in kablarna mellan DE10-Lite-kortet pinnes och ultraljudsensor HC-SR05.

5.1 Systemarkitektur

TOPPNIVÅN arkitektur visas i Figur 3, i figur 4 visar komponenthierarki och i figur 5 visar QSYS interface.



Figur 3. Systemarkitekturen för TEIS ulraljudsensor



Figur 4. Komponenthierarki för ultraljudsensor



Figur 5. QSYS interface för IP komponenter av ultraljudsensor

5.2 Delsystem

Systemet innehålla följande komponenter:

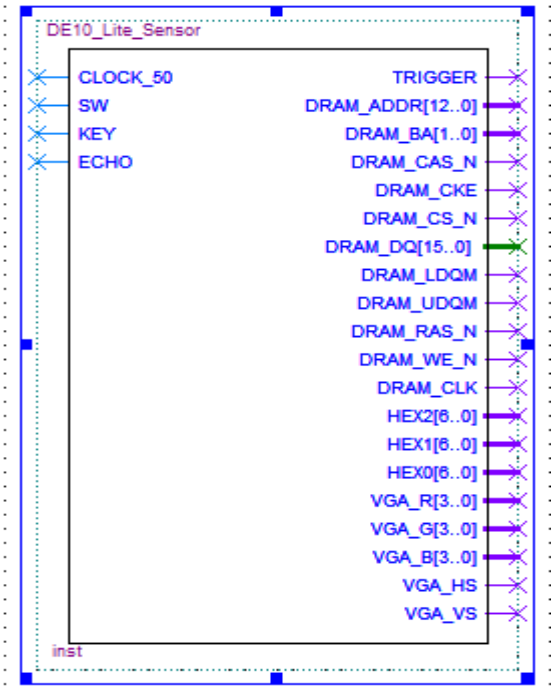
5.2.1 Toppnivån

Komponent namn: ***DE10_Lite_Sensor***

5.2.1.1 Systemarkitektur

Den toppnivån användas för att instansierar Qsys interface son innehålla hela IP komponenterna. Toppnivån får insignaler och skickar utsignaler till sjusegmentdisplayerna för att visa mätningsresultat och skicka utsignaler till skärmen för att rita olika färger för olika avstånd.

TOPPNIVÅN symbol visas i Figur 6 och i Tabell 3 visar In- och utsignalerna.



Figur 6. TOPPNIVÅN för ultraljudsensor

Tabell 3. In- och utsignalerna till TOPP filen

Signal	**Namn**	**Riktning**	**Typ**
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	SW	in	std_logic
Pio in_key	KEY	in	std_logic
Echo timer	ECHO	in	std_logic
Trigger-10us puls	TRIGGER	out	std_logic
External minne	SDRAM	out	std_logic, std_logic_vector
7-segmentsdisplay till hundratal	undratal	out	std_logic_vector(6 downto 0)
7-segmentsdisplay till tiotal	tiotal	out	5 std_logic_vector(6 downto 0)
7-segmentsdisplay till ental	ental	out	std_logic_vector(6 downto 0)
Horizontell synkronism	vga_hs	out	std_logic
Vertikal synkronism	vga_vs	out	std_logic
Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)
Blå signal	vga_b	out	std_logic_vector(3 downto 0)

Ingången signaler

CLOCK_50: 50Mhz input clock.

sw(reset_n): Reset av signalen. Active low.

echo: Pin som aktiverar sig för att börjar mäta avståndet. ”1” det börjar att räknar, ”0” slutar att räknar.

Key: skifta mellan presentations skärm och avstånd mäta skärmen.

Utgången signaler

SDRAM: Externa minne som innehålla data och instruktioner.

trigger: Pin som får aktiveringen puls efter 10us för att kan börjar avståndsmätare.

undratal: visa 7-segmentsdisplay till hundratal.

tiotal: visa 7-segmentsdisplay till tiotal.

ental: visa 7-segmentsdisplay till ental.

vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.

vga_vs: Vertikal synkronism. Kontrollerar början och slutet på en ram(framme) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.

vga_r: Definierar röd färg av signalen.

vga_g: Definierar grön färg av signalen.

vga_b: Definierar blå färg av signalen.

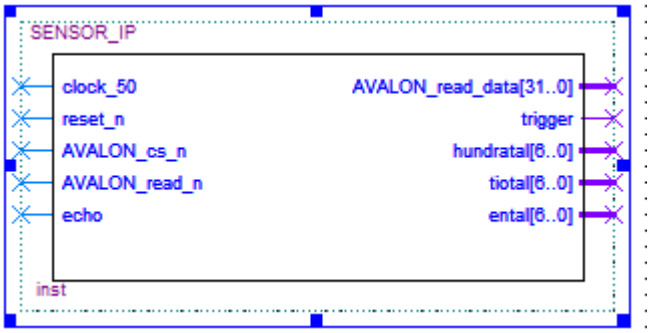
5.2.2 Sensor IP – komponent

Komponent namn: ***SENSOR_IP***

5.2.2.1 Systemarkitektur

CPU'n read genom denna ip_komponent med en läs process av busen Avalon för att få avståndet även instansierar hela komponenterna som används för att utföra avståndsmätare med ultraljudsensor HC-SR05.

SENSOR_IP-komponentens symbol visas i Figur 7 och i Tabell 4 visar In- och utsignalerna.



Figur 7. SENSOR_IP för ultraljudsensor

Tabell 4. In- och utsignalerna till SENSOR_IP filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Avalon chip selsect	Avalon_cs_n	in	std_logic
Avalon läsning	Avalon_read_n	in	std_logic
Echo avstånd	echo	in	std_logic
Trigger-10us puls	trigger	out	std_logic
Data läsning	Avalon_read_data	out	std_logic_vector(31 downto 0)
7-segmentsdisplay till hundratal	hundratal	out	std_logic_vector(6 downto 0)
7-segmentsdisplay till tiotal	tiotal	out	std_logic_vector(6 downto 0)
7-segmentsdisplay till ental	ental	out	std_logic_vector(6 downto 0)

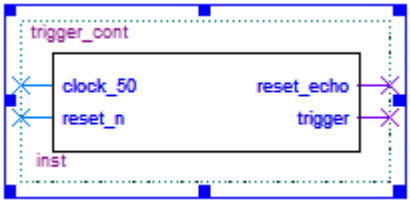
5.2.3 Trigger - komponent

Komponent namn: ***trigger_cont***
*Instansnamn****: b1v_inst_trigger_cont***

5.2.3.1 Systemarkitektur

Trigger pin av sensorn får en puls av minst 10us från FPGA, igenom denna puls indikerar till sensorn att kan börjar avståndsmätare.

TRIGGER-komponentens symbol visas i Figur 8 och i Tabell 5 visar In- och utsignalerna.



Figur 8. TRIGGER 10us för ultraljudsensor

Tabell 5. In- och utsignalerna till TRIGGER filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Echo reset	reset_echo	out	std_logic
Trigger-10us puls	trigger	out	std_logic

Ingången signaler
CLOCK_50: 50Mhz input clock.
reset_n: reset av signalen. Active low.

Utgången signaler
reset_echo: efter 10us det blir reset=’0’ för att initierar echo räknaren.
trigger: Pin som får aktiveringen puls efter 10us för att kan börjar avståndsmätare.

5.2.4 Echo - komponent

Komponent namn: ***echo_cont***
*Instansnamn****: b1v_inst_echo_cont***

5.2.4.1 Systemarkitektur

Igenom echo pin, den sensorn återvänder till FPGA en puls vars bredd är proportionell till tiden som ljud tar att resa från sensorn till hinder och sen återgår till sensorn igen.

ECHO-komponentens symbol visas i Figur 9 och i Tabell 6 visar In- och utsignalerna.



Figur 9. ECHO räknaren för ultraljudsensor

Tabell 6. In- och utsignalerna till ECHO filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Echo reset	reset_echo	in	std_logic
Echo enable	enable_echo	in	std_logic
Nro. klocks echo	klocks_echo	out	std_logic_vector(23 downto 0)

Ingången signaler
CLOCK_50: 50Mhz input clock.
reset_n: reset av signalen. Active low.
reset_echo: efter 10us det blir reset=’0’ för att initierar echo räknaren.
Enable_echo: Startar räknaren av antal klockor på stigande flank '1' och slutar räknaren av antal klockor på fallande flank '0'.

Utgången signaler
klocks_echo: ökar puls-klockor när enable_echo='1'.

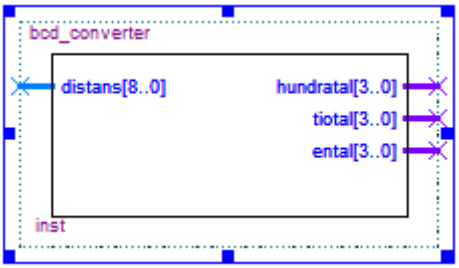
5.2.5 Bcd konverter - komponent

Komponent namn: ***bcd_converter***
*Instansnamn****: b1v_inst_bcd_converter***

5.2.5.1 Systemarkitektur

Denna komponent får avståndet i binär för att sen omvandlar i BCD för att kan visa på 7-segment display. Det använder metoden flyttar till vänster och adderar 3(shift-and-add-3).

BCD CONVERTER-komponentens symbol visas i Figur 10 och i Tabell 7 visar In- och utsignalerna.



Figur 10. Omvandlar binär till BCD

Tabell 7. In- och utsignalerna till BCD_CONVERTER filen

Signal	Namn	Riktning	Typ
Avstånd	distans	in	std_logic_vector(8 downto 0)
Bcd om bas 10	hundratal	out	std_logic_vector(3 downto 0)
Bcd om bas 10	tiotal	out	std_logic_vector(3 downto 0)
Bcd om bas 10	ental	out	std_logic_vector(3 downto 0)

Ingången signaler
distans: avstånd i centimeter.

Utgången signaler
undratal: bcd utgången som hundratal på bas 10.
tiotal: bcd utgången som tiotal på bas 10.
ental: bcd utgången som ental på bas 10.

5.2.6 Sju_seg_displayer - komponent

Komponent namn: ***sju_displayer_komponent***
*Instansnamn****: b2v_inst_sju_displayer_komponent_1,***
b2v_inst_sju_displayer_komponent_2,
b2v_inst_sju_displayer_komponent_3

5.2.6.1 Systemarkitektur

Denna komponent styrprogram för 7-segment display för att visar avståndet.

SJU SEGMENT DISPLAYER-komponentens symbol visas i Figur 11 och i Tabell 8 visar In- och utsignalerna.

Figur 11. Sjusegmentdisplayer



Tabell 8. In- och utsignalerna till SJU_SEG_DISPLAYER filen

Signal	Namn	Riktning	Typ
Avstånd på bas 10	tal_in	in	std_logic_vector(3 downto 0)
Styrdata till HEX6	hex_out	out	std_logic_vector(6 downto 0)

Ingången signaler
tal_in: binärt tal (0-15).
Utgången signaler
hex_out: 7-segment display.

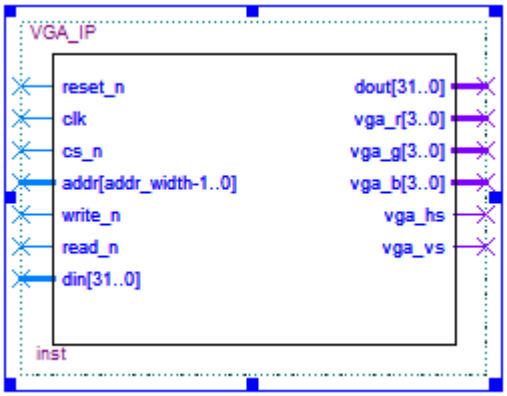
5.2.7 Vga IP – komponent

Komponent namn: **VGA_IP**

5.2.7.1 Systemarkitektur

CPU'n genom denna ip_komponent läs eller skriv i busen Avalon för att skickar information till skärmen.

VGA IP-komponentens symbol visas i Figur 12 och i Tabell 9 visar In- och utsignalerna.



Figur 12. Sjusegmentdisplayer

Tabell 9. In- och utsignalerna till VGA_IP filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLK	in	std_logic
Switch reset	reset_n	in	std_logic
Chipselect	cs_n	in	std_logic
Adress bus in	addr	In	std_logic_vector(16 downto 0)
Write enable	write_n	In	std_logic
Read enable	read_n	In	std_logic
Data bus in	din	In	std_logic_vector(31 downto 0)
Data bus out	dout	out	std_logic_vector(31 downto 0)
Horizontell synk	vga_hs	out	std_logic
vertikal synk	vga_vs	out	std_logic
Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)
Blå signal	vga_b	out	std_logic_vector(3 downto 0)

Ingången signaler
CLK: 50Mhz input clock.
reset_n: Reset av signalen. Active low.
Chipselect: Active low.
addr: skriver eller läser på komponenten.
write_n: aktiverar skrivning på Avalon interface.
read_n: aktiverar läsning på Avalon interface
din: Data bus ingång som skriver till komponent.

Utgången signaler
dout: Data bus utgång som läser från komponent.
vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.
vga_vs: Vertikal synkronism. Kontrollerar början och slutet på en ram (framme) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.
vga_r: Definierar röd färg av signalen.
vga_g: Definierar grön färg av signalen.
vga_b: Definierar blå färg av signalen.

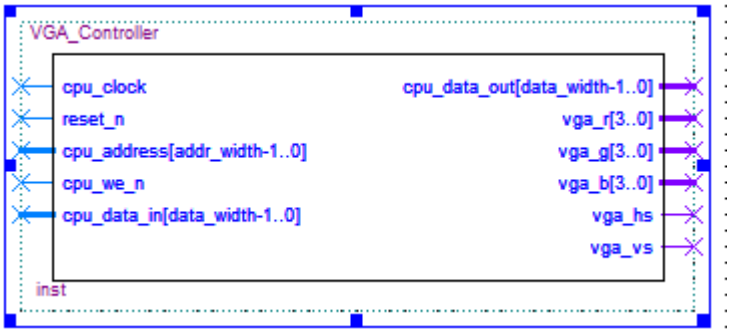
5.2.8 Vga kontroller - komponent

*Komponent namn: **vga_controller***
*Instansnamn **vga_controller_inst***

5.2.8.1 Systemarkitektur

Den komponent användas för att räknar en skärm av 640 x 480 pixel för att kan aktivera horisontell, vertikal och mörk synkronisering även skapa adressen signal(addr_sync) som kommer att använda på grafikminnet. addr_sync är för 320 (pixel på en rad) * 240 (rader) upplösning.

VGA CONTROLLER-komponentens symbol visas i Figur 13 och i Tabell 10 visar In- och utsignalerna.



Figur 13. VGA kontroller för ultraljudsensor

Tabell 10. In- och utsignalerna till VGA_CONTROLLER filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	cpu_clock	in	std_logic
Switch reset	reset_n	in	std_logic
Adress bus in	cpu_address	In	std_logic_vector(16 downto 0)
Skriver på ram	cpu_we_n	In	std_logic
Data bus in	cpu_data_in	In	std_logic
Data bus out	cpu_data_out	out	std_logic
Horizontell synk	vga_hs	out	std_logic
Vertikal synk	vga_vs	out	std_logic
Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)
Blå signal	vga_b	out	std_logic_vector(3 downto 0)

Ingången signaler

cpu_clock: 50Mhz input clock.
reset_n: reset av signalen. Active low.
Cpu_address: skriver eller läser på komponenten.
cpu_we_n: aktiverar skrivning på Avalon interface.
cpu_data_in: Data bus ingång som skriver till minnet.

Utgången signaler

cpu_data_out: Data bus utgång som läser från minnet.

vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.

vga_vs: Vertikal synkronism. Kontrollerar början och slutet på en ram(framme) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.

vga_r: Definierar röd färg av signalen.

vga_g: Definierar grön färg av signalen.

vga_b: Definierar blå färg av signalen.

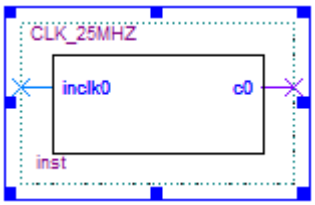
5.2.9 PLL – komponent

Komponent namn: ***clk_25mhz***
Instansnamn ***clk_25_mhz_inst***

5.2.9.1 Systemarkitektur

Den komponent användas för att dela 50Mhz klockan till 25Mhz för vga_controller och grafikminnet.

PLL komponent symbol visas i Figur 14 och tabell 11 visar In- och utsignalerna.



Figur 14. Klockan 25mhz

Tabell 11. In- och utsignalerna till CLK_25MHZ filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	inclk0	in	std_logic
Switch reset	reset_n	in	std_logic
Klocksignal 25 MHz	c0	out	std_logic

Ingången signaler
Inclk0: 50Mhz input clock.

Utgången signaler
c0: 25Mhz input klock för VGA_controller och grafikminnet.

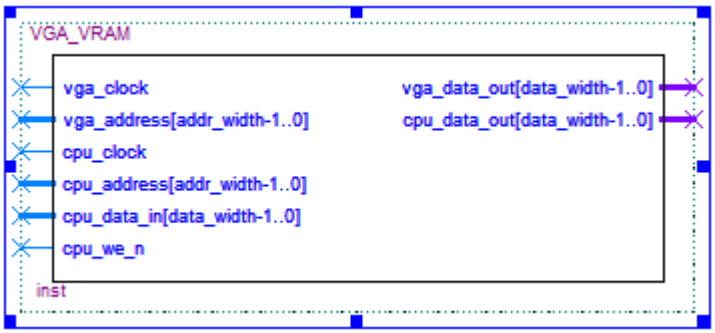
5.2.10 Ram kontroller - komponent

Komponent namn: ***vga_ram***
Instansnamn ***vga_ram_inst***

5.2.10.1 Systemarkitektur

Den komponenten används för att skriver och läsa data på minnet.

VGA RAM-komponentens symbol visas i Figur 15 och i Tabell 12 visar In- och utsignalerna.



Figur 15. VGA kontroller för ultraljudsensor

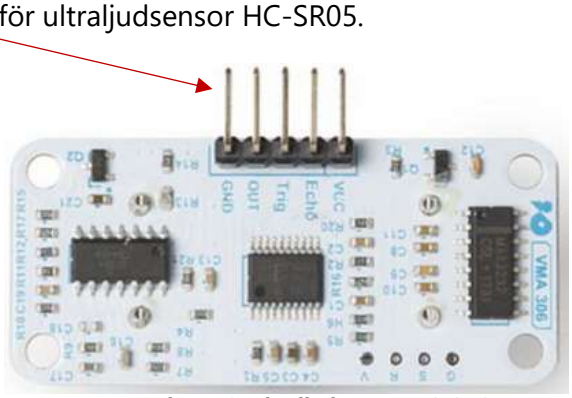
Tabell 12. In- och utsignalerna till VGA_CONTROLLER filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	cpu_clock	in	std_logic
Klocksignal 25 MHz	vga_clock	in	std_logic
Address till vga	vga_address	In	std_logic_vector(16 downto 0)
Address till minne	cpu_address	in	std_logic_vector(16 downto 0)
Aktiverar skrivning	cpu_we_n	In	std_logic
Data bus in	cpu_data_in	In	std_logic_vector(2 downto 0)
Data bus till skärmen	vga_data_out	out	std_logic_vector(2 downto 0)
Data bus från minnet	cpu_data_out	out	std_logic_vector(2 downto 0)

5.3 Ultraljudsensor hc-sr05

För att utveckla projektet det använd avståndsmätaren med ultraljud för Arduino. Når upp till 4,5 m med 3 mm noggrannhet (15° vinkel). Drivspänning: 5 V. Strömförbrukning: 2 mA. Frekvens: 40 kHz. Modell: HC-SR05.

I Figur 16 visar pinar för ultraljudsensor HC-SR05.



Figur 16. Ultraljudsensor HC-SR05

5.4 I/O pinnar

Sensoranslutningen med DE10-lite kort är mycket enkel. Vi kan göra det med en protoboard eller direkt med ledningar. För att få sensorn att fungera krävs 4 signaler:

- VCC : får 5V från DE10-lite kort
- GND : GND
- ECHO : Input. När echo är positive flank börjar timer.
- TRIGGER : Output aktivering. 10us digital pulse.
- OUT : Det använder inte på projektet.

Figur 17 visar pinnar för DE10-Lite kortet.

Node Name	Direction	Location
altera_reserved_tck	Input	
altera_reserved_tdi	Input	
altera_reserved_tdo	Output	
altera_reserved_tms	Input	
CLOCK_50	Input	PIN_N14
DRAM_ADDR[12]	Output	PIN_R20
DRAM_ADDR[11]	Output	PIN_P20
DRAM_ADDR[10]	Output	PIN_T20
DRAM_ADDR[9]	Output	PIN_P19
DRAM_ADDR[8]	Output	PIN_P18
DRAM_ADDR[7]	Output	PIN_R18
DRAM_ADDR[6]	Output	PIN_T19
DRAM_ADDR[5]	Output	PIN_T18
DRAM_ADDR[4]	Output	PIN_U19
DRAM_ADDR[3]	Output	PIN_U18
DRAM_ADDR[2]	Output	PIN_V18
DRAM_ADDR[1]	Output	PIN_W19
DRAM_ADDR[0]	Output	PIN_U17
DRAM_BA[1]	Output	PIN_T22
DRAM_BA[0]	Output	PIN_T21
DRAM_CAS_N	Output	PIN_U21
DRAM_CKE	Output	PIN_N22
DRAM_CLK	Output	PIN_L14
DRAM_CS_N	Output	PIN_U20
DRAM_DQ[15]	Bidir	PIN_F22
DRAM_DQ[14]	Bidir	PIN_G19
DRAM_DQ[13]	Bidir	PIN_G20
DRAM_DQ[12]	Bidir	PIN_G22
DRAM_DQ[11]	Bidir	PIN_H22
DRAM_DQ[10]	Bidir	PIN_H21
DRAM_DQ[9]	Bidir	PIN_J22
DRAM_DQ[8]	Bidir	PIN_P21

Node Name	Direction	Location
DRAM_DQ[7]	Bidir	PIN_V21
DRAM_DQ[6]	Bidir	PIN_W20
DRAM_DQ[5]	Bidir	PIN_W22
DRAM_DQ[4]	Bidir	PIN_Y22
DRAM_DQ[3]	Bidir	PIN_AA21
DRAM_DQ[2]	Bidir	PIN_AA22
DRAM_DQ[1]	Bidir	PIN_Y20
DRAM_DQ[0]	Bidir	PIN_Y21
DRAM_LDQM	Output	PIN_V22
DRAM_RAS_N	Output	PIN_U22
DRAM_UDQM	Output	PIN_J21
DRAM_WE_N	Output	PIN_V20
ECHO	Input	PIN_AB17
HEX0[6]	Output	PIN_C17
HEX0[5]	Output	PIN_D17
HEX0[4]	Output	PIN_E16
HEX0[3]	Output	PIN_C16
HEX0[2]	Output	PIN_C15
HEX0[1]	Output	PIN_E15
HEX0[0]	Output	PIN_C14
HEX1[6]	Output	PIN_B17
HEX1[5]	Output	PIN_A18
HEX1[4]	Output	PIN_A17
HEX1[3]	Output	PIN_B16
HEX1[2]	Output	PIN_E18
HEX1[1]	Output	PIN_D18
HEX1[0]	Output	PIN_C18
HEX2[6]	Output	PIN_B22
HEX2[5]	Output	PIN_C22
HEX2[4]	Output	PIN_B21
HEX2[3]	Output	PIN_A21
HEX2[2]	Output	PIN_B19
HEX2[1]	Output	PIN_A20
HEX2[0]	Output	PIN_B20
KEY	Input	PIN_B8
SW	Input	PIN_F15
TRIGGER	Output	PIN_AA17
VGA_B[3]	Output	PIN_N2
VGA_B[2]	Output	PIN_P4
VGA_B[1]	Output	PIN_T1
VGA_B[0]	Output	PIN_P1
VGA_G[3]	Output	PIN_R1
VGA_G[2]	Output	PIN_R2
VGA_G[1]	Output	PIN_T2
VGA_G[0]	Output	PIN_W1
VGA_HS	Output	PIN_N3
VGA_R[3]	Output	PIN_Y1
VGA_R[2]	Output	PIN_Y2
VGA_R[1]	Output	PIN_V1
VGA_R[0]	Output	PIN_AA1
VGA_VS	Output	PIN_N1

Figur 17. Pinnar för DE10-Lite kortet

6. ANALYS

6.1 Frekvens och storlek före optimering

Denna del visar max frekvens för hela konstruktionen och varje delsystem.

6.1.1 Max frekvens för hela konstruktionen

Denna konstruktion är kompilerade i en 50 MHZ som systemklocka och har det en klocka av 25 MHZ(CLK_25) för att köras med VGA skärmen. Resurser som används i konstruktionen det passade bra på VGA-kortet men kan det optimerar sig bättre med advisor verktygen.

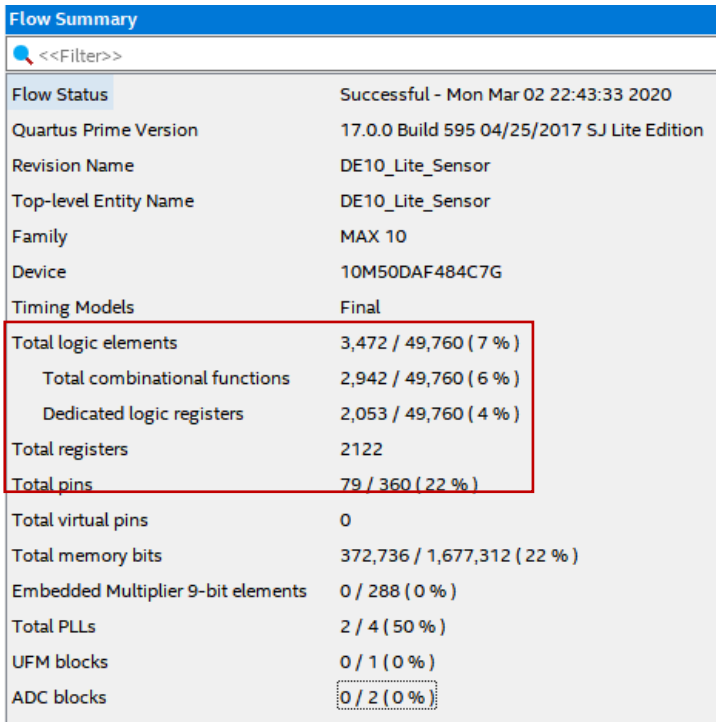
I Tabell 13 visar max frekvens för hela konstruktionen.

Tabell 13: Max frekvens av konstruktionen.

Modell	**Fmax systemklocka**
Slow 1200mV 85C Model	91.42 MHz
Slow 1200mV 0C Model	99.69 MHz
Modell	**Fmax CLK_25**
Slow 1200mV 85C Model	108.61 MHz
Slow 1200mV 0C Model	118.08 MHz

6.1.2 Storlek för hela konstruktionen och delsystem

Figur 18 visar konstruktionens resursanvändning, figur 19 visar delsystem resursanvändning.



Figur 18: Konstruktionens resursanvändning.

I Figur 19 visar resursanvändning för varje delsystem. Nummer som är utanför parentes indikerar antal av resurser som kan använda varje entity och nummer som är utanför parentes indikerar antal av resurser som används varje entity.

Analysis & Synthesis Resource Utilization by Entity

<<Filter>>

	Compilation Hierarchy Node	Combinational ALUTs	Dedicated Logic Registers	Memory Bits
1	|DE10_Lite_Sensor	2928 (1)	2096 (0)	372736
1	|DE10_Lite_System...DE10_Lite_System|	2775 (0)	2011 (0)	372736
1	|DE10_Lite_System_cpu:cpu|	1007 (0)	585 (0)	10240
2	|DE10_Lite_System...g_uart:jtag_uart|	139 (36)	113 (13)	1024
3	|DE10_Lite_Syste..._interconnect_0|	915 (0)	894 (0)	0
4	|DE10_Lite_Syste...am_controller_0|	275 (218)	248 (156)	0
5	|DE10_Lite_Syste..._ram:onchip_ram|	1 (1)	0 (0)	131072
6	|DE10_Lite_Syste..._key:pio_in_key|	1 (1)	1 (1)	0
7	|DE10_Lite_System_pll_0:pll_0|	8 (7)	6 (2)	0
8	|SENSOR_IP:sensor_ip_0|	190 (36)	79 (31)	0
9	|TIMER_HW_IP:timer_hw_ip_0|	66 (2)	34 (2)	0
10	|VGA_IP:de10_lite_vga_ip_0|	166 (3)	32 (4)	230400
11	|de10_lite_system_...er:rst_controller|	1 (0)	3 (0)	0
12	|de10_lite_system...t_controller_001|	6 (0)	16 (0)	0
2	|sld_hub:auto_hub|	152 (1)	85 (0)	0

Figur 19: Delsystem resursanvändning.

6.2 Effektförbrukning uppskattning

PowerPlay Power Analyzer Summary

<<Filter>>

PowerPlay Power Analyzer Status	Successful - Mon Mar 02 23:49:01 2020
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite Edition
Revision Name	DE10_Lite_Sensor
Top-level Entity Name	DE10_Lite_Sensor
Family	MAX 10
Device	10M50DAF484C7G
Power Models	Final
Total Thermal Power Dissipation	227.69 mW
Core Dynamic Thermal Power Dissipation	52.72 mW
Core Static Thermal Power Dissipation	102.95 mW
I/O Thermal Power Dissipation	72.02 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

Figur 20: Effektförbrukning med toggle rate på 12,5% och typiska driftförhållanden.

6.3 Tisdvalidering

Timingkraven definierad i SDC-filen möttes, inga röda siffror rapporterades i TimeQuest. Den kortaste slacktiden visas i Figur 21.

Slow 1200mV 85C Model

Command: Summary of Paths

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	9.062	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[22]	DE10_Lit...|d_write	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.110	10.741
2	9.064	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[22]	DE10_Lit...accepted	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.110	10.739
3	9.080	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[21]	DE10_Lit...|d_write	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.110	10.723
4	9.082	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[21]	DE10_Lit...accepted	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.110	10.721
5	9.222	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[23]	DE10_Lit...|d_write	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.110	10.581
6	9.224	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[23]	DE10_Lit...accepted	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.110	10.579
7	9.260	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[22]	DE10_L..._from_R	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.109	10.544
8	9.275	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[22]	DE10_Lit...|W_valid	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.109	10.529
9	9.278	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[21]	DE10_L..._from_R	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.109	10.526
10	9.293	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[21]	DE10_Lit...|W_valid	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.109	10.511
11	9.332	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[20]	DE10_Lit...|d_write	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.110	10.471
12	9.334	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[20]	DE10_Lit...accepted	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.110	10.469
13	9.363	DE10_Lite_System:b0v_inst_DE1..._cpu_cpu:cpu|W_alu_result[24]	DE10_Lit...|d_write	SYSTEM_CLOCK	SYSTEM_CLOCK	20.000	-0.106	10.444

Figur 21: Timingrapport från TimeQuest.

7. VERIFIERING AV TESTPROTOKOLL

Tabell 14. Testprotokoll - verifiering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	Gör en reset av systemet. Hela skärmen skall rensas vid uppstart av programmet.	Funktionen clearScreen(0) används för att skriva färgvärdet svart (0) till hela grafikminnet.	Ja
2	Presentation av projektet visas efter att skärmen rensats vid uppstart.	Introduktionsskärm med en kort presentation visas efter uppstart.	Ja
3	Knapp 0 skall skifta mellan "presentations"-läge och "loggnings"-läge.	Skärmen visar presentations"-läge eller "loggnings"-läge.	Ja
4	Det visar avståndet efter trycker knapp 0	Visar avstånden siffror på skärmen	Ja
5	Det visar cirklar	Det visar 3 cirklar som ändrar färger när ändrar avstånden.	Ja

8. VALIDERING KONSTRUKTIONEN

8.1 Testprotokoll för validering

I Tabell 15 visar testprotokoll för validering och figur 20 visar validering i VGA-skärmen.

Tabell 15. Testprotokoll för validering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	Avstånd <=10 cm	- Visar centimeter på sjusegmentdisplay - Visar centimeter och röd färg på skärmen	Ja(se figur 22)
2	Avstånd >10 cm och <=20	- Visar centimeter på sjusegmentdisplay - Visar centimeter och grön färg på skärmen	Ja(se figur 23)
3	Avstånd >20 cm och <=30	- Visar centimeter på sjusegmentdisplay - Visar centimeter och blå färg på skärmen	Ja(se figur 24)
4	Avstånd >30	- Visar centimeter på sjusegmentdisplay - Visar centimeter och vit färg på skärmen	Ja(se figur 25)



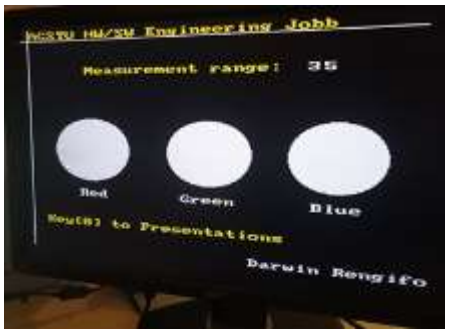
Figur 22: visa röd färg.



Figur 23: visa grön färg.



Figur 24: visa röd färg.



Figur 25: visa vit färg.

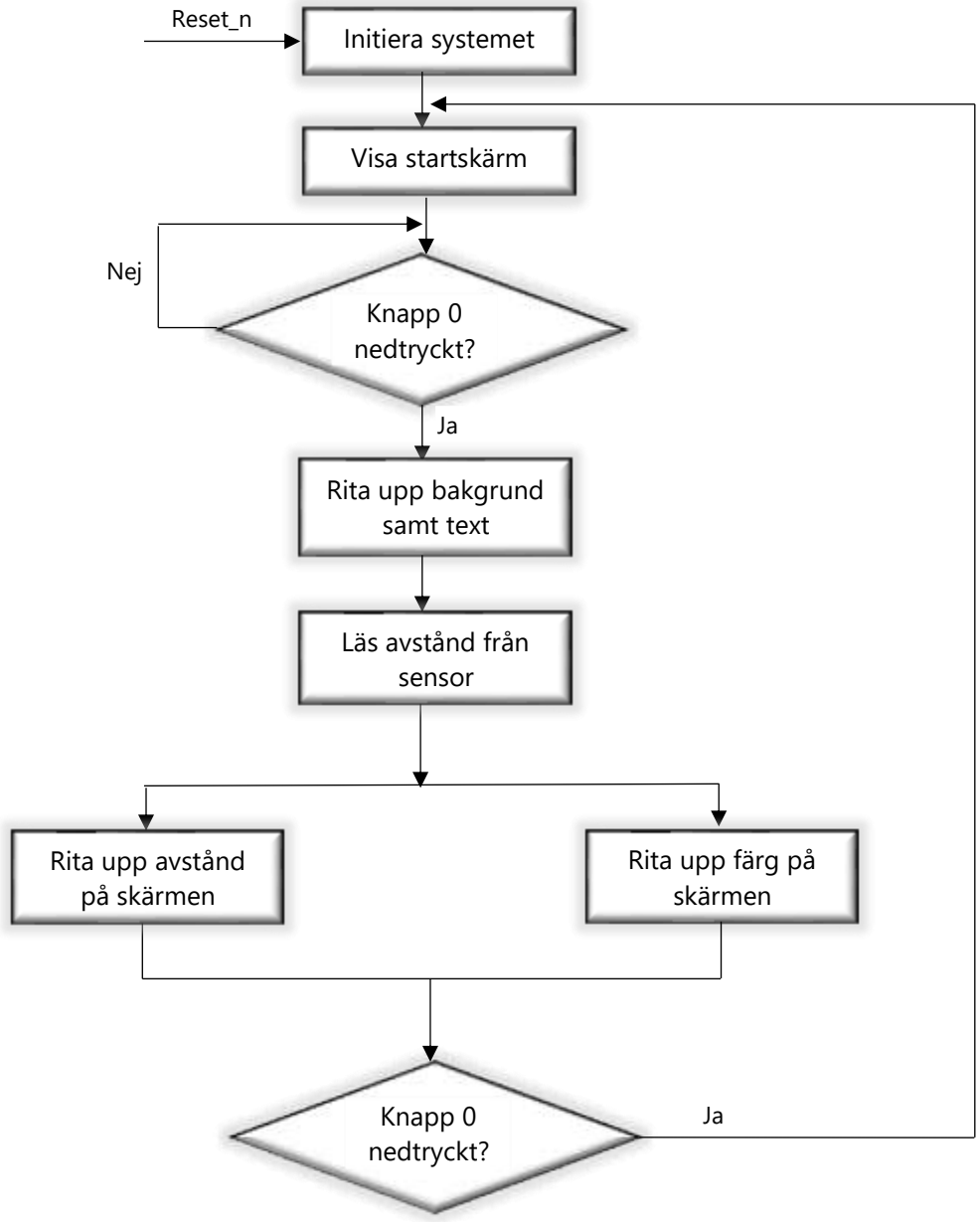
8.2 Validering film

https://www.youtube.com/watch?v=KcT34QG7Ulg

9. MJUKVARAN ARKITEKTUR

9.1 Systemarkitektur

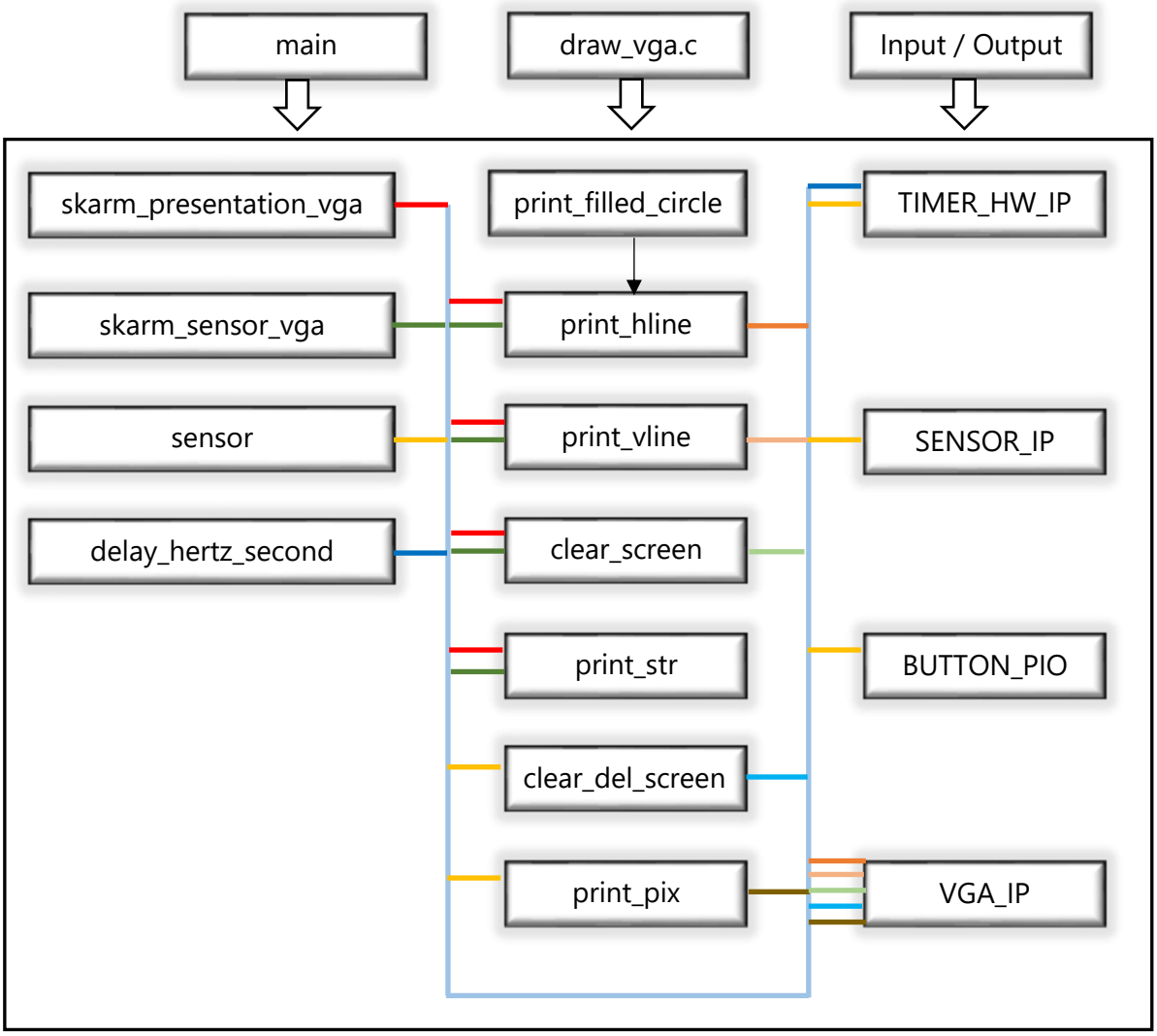
Flödesschemat för mjukvaran visas i Figur 26. När systemet börjar, visas en startskärm som beskriver hur man kontrollerar systemet. Efter att man tryckt på knapp 0 startar sedan en kontinuerlig sampling och uppdatering av avståndet på skärmen.



Figur 26. Flödesdiagram

9.2 Delsystem

Huvudprogrammet innehåller main och de funktioner som hanterar loggning av data, tidräkning och bildhantering. Funktionerna i draw_vga.c används av huvudprogrammet för att utföra de grundläggande grafik funktionerna som till exempel att tända pixlar, rita linjer och skriva tecken på skärmen. Grafikrutinerna använder i sin tur vga utgångens IP-komponent till att skriva och läsa i grafikminnet.



9.2.1 Minneskarta

Memory-map från ”sumary.html” för konstruktionen, se figur 27.

Slave Descriptor	Address Range	Size	Attributes
SENSOR_IP_0	0x08089040 - 0x08089043	4	
jtag_uart	0x08089038 - 0x0808903F	8	printable
sysid	0x08089030 - 0x08089037	8	
TIMER_HW_IP_0	0x08089020 - 0x0808902F	16	
pll_0	0x08089010 - 0x0808901F	16	
pio_in_key	0x08089000 - 0x0808900F	16	
onchip_ram	0x08084000 - 0x08087FFF	16384	memory
DE10_Lite_VGA_IP_0	0x08000000 - 0x0807FFFF	524288	
new_sdram_controller_0	0x04000000 - 0x07FFFFFF	67108864	memory

Figur 27. Flödesdiagram

9.3 Drivrutiner

Det visar en översikt över de funktioner som krävdes av kravspecifikationen.

9.3.1 VGA färger

Värde (hexadecimalt)	Färg
0x00	
0x01	
0x02	
0x03	
0x04	
0x05	
0x06	
0x07	

9.3.2 VGA-drivrutiner

Tabell 16: "DE10_Lite_VGA_Driver.h" innehåller funktionsdeklarationer och inkluderas för att få tillgång till vga-rutinerna.

Tabell 17: ***"draw_vga.c"*** innehåller vga-rutinerna.

Tabell 18: ***"DE10_Lite_VGA_Driver.h"*** innehåller också definitioner av macron som fungerar för att läser och skriver värden i minnesplacering.

Tabell 16. Funktionsdeklarationer i DE10_Lite_VGA_Driver.h

Funktion	Beskrivning
void print_pix(alt_u32 x, alt_u32 y, alt_u8 rgb);	Skriver ut en pixel på koordinaterna x,y med färgen rgb.
void print_hline(alt_u32 x_start, alt_u32 y_start, alt_u32 length, alt_u8 rgb);	Skriver ut en horisontell linje med färgen rgb. Linjen börjar på x_start,y_start och är length pixlar

	lång.
void print_vline(alt_u32 x_start, alt_u32 y_start, alt_u32 length, alt_u8 rgb);	Skriver ut en vertikal linje med färgen rgb. Linjen börjar på x_start,y_start och är length pixlar lång.
void print_char(alt_u32 x, alt_u32 y, alt_u8 rgb, alt_u8 bg_rgb, alt_u8 ch);	Skriver ut ett tecken, ch, på x,y. Tecknet har färgen rgb och bakgrundsfärgen bg_rgb.
alt_u8 read_pixel_ram_int(alt_u32 x, alt_u32 y);	Läser av värdet på pixeln på koordinaten x,y, det vill säga färgvärdet.
void print_filled_circle(alt_32 x0, alt_32 y0, alt_32 radius, alt_32 color);	Skriver ut en cirkel med radien radius och färgen rgb. Centrum är x,y.
void clear_screen(alt_u8 rgb);	Rensar skärmen med färgen rgb.
void print_str(alt_u32 x, alt_u32 y, alt_u8 rgb, alt_u8 bg_rgb, alt_u8 *str);	Skriver ut en sträng på koordinaterna x,y med färgen rgb och bakgrundsfärgen bg_rgb. Skulle strängen fortsätta utanför högerkanten fortsätter den på raden under. Skulle den gå utanför nederkanten fortsätter den längst upp till vänster.
void int_print(alt_32 x_start, alt_32 y_start, int data, int data_l, alt_32 color, alt_u32 BGcolor)	Skriver ut siffror(data) med antal siffror(data_l) på koordinaterna x,y med färgen color och bakgrundsfärgen BGrgb.

Tabell 17. VGA-rutinerna i draw_vga.c

Funktion	Beskrivning
print_pix(82,72,0x02) ;	i kolumn,rad skriver ut pixeln med färgen grön.
print_hline(0,14,282, 0x07);	ritar en 282 pixlar lång horisontell vit linje med start uppe i vänstra.
print_vline(7,4,220, 0x07);	ritar en 220 pixlar lång vertikalvit linje med start uppe i vänstra hörnet.
Print_char(10, 20, 0x04, 0x07, 'D');	skriver ut bokstaven ”D” i röd med vit bakgrund högst upp i vänstra hörnet.
alt_u8 var = read_pixel_ram_int(0,0);	returnerar färg koden på pixeln längst uppe i vänstra.
print_circle(40, 50, 50, 1);	ritar och fyller i en blå cirkel med en radie på 40 pixlar i övre vänstra hörnet av skärmen.
clear_screen(0x00);	skriver över alla adresser i bild minnet med färgen black.

print_str(8, 6, 1, ”TEIS”, 0x04,0);	skriver ut texten ”TEIS” i röd färg högst upp i vänstra hörnet på skärmen.
int_print(40,65,123456,3,0x02,0)	Skriver ut tre heltal med färgen grön

Tabell 18. VGA-macron i DE10_Lite_VGA_Driver.h

Funktion	Beskrivning
#define write_pixel(x,y,color) IOWR_32DIRECT(DE10_LITE_VGA_IP_0_BASE,((320*(y) +(x))*4),(color))	Skriver värde ”color” till position x_pos,y_pos i minnet.
#define read_pixel(x,y) IORD_32DIRECT(DE10_LITE_VGA_IP_0_BASE,(320*(y)+ (x))*4)	Läser värde från x_pos, y_pos i minnet.

9.3.3 Timmer-drivrutiner

För att använda timer-drivrutinen finns de macron som är listade i Tabell 19.

Tabell 19. Timmer-drivrutiner i altera_avalon_timer_regs.h

Funktion	Beskrivning
#define TIMER_STOP IOWR_32DIRECT(TIMER_HW_IP_0_BASE,4,0x00000000)	Stoppar timern
#define TIMER_RESET IOWR_32DIRECT(TIMER_HW_IP_0_BASE,4,0x40000000)	Nollställer timern
#define TIMER_START IOWR_32DIRECT(TIMER_HW_IP_0_BASE,4,0x80000000)	Startar timern
#define TIMER_READ IORD_32DIRECT(TIMER_HW_IP_0_BASE,0)	Läser ut timervärdet

9.3.4 Sensor-drivrutiner

Tabell 20: ”SENSOR_regs.h” innehåller definitioner av macron som fungerar för att läser och skriver värden i minnesplacering.

Tabell 20. Funktionsdeklarationer i SENSOR_regs.h

Funktion	Beskrivning
#define SENSOR_WRITE(base,data) IOWR_32DIRECT(SENSOR_IP_0_BASE, 0,data)	Skriver värde i minnet.
#define SENSOR_READ(base) IORD_32DIRECT(SENSOR_IP_0_BASE, 0)	Läser värde från minnet.

10. OPTIMERING

Som synes i tabell 21 anpassades BSP:n för att bli så effektiv och snabb som möjligt.

Tabell 21. BSP-Optimeringsparametrar använda.

Funktion	Tidsåtgång (offset = 50mhz) (cc*1)/50mhz
enable_small_c_library	På
enable_reduced_device_drivers	På
enable_lightweight_device_driver_api	På
enable_exit	Av
enable_clean_exit	Av
enable_c_plus_plus	Av
Optimize	Size

11. REFERENSER

- Sensorn HC-SR04: https://www.geekfactory.mx/tutoriales/tutoriales-arduino/sensor-ultrasonico-hc-sr04-y-arduino/ : Arduino tutorials [16 maj 2014]

- BCD convert : https://en.wikipedia.org/wiki/Double_dabble : Wikipedia [redigerades senast den 21 oktober 2019]

- https://geekelectronica.com/medicion-de-distancia-con-sensor-ultrasonico-y-arduino/ : Geek Electrónica [5 maj 2016]